Technical report — SwissDoc Q&A

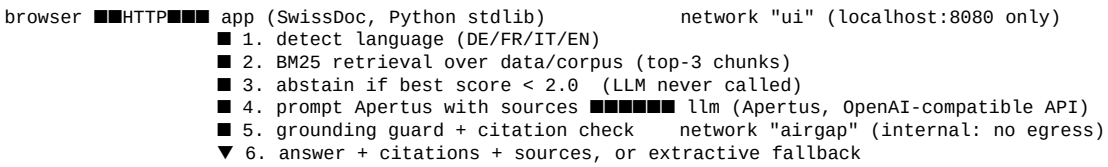
An air-gapped, multilingual question-answering assistant for Swiss public-service information, powered by Apertus.

- **Track:** Track 2B — SwissDoc Q&A (Own Project)
- **Event:** Online
- **Team:** AinarsHibners — Ainars Hibners
- **Demo:** `docs/demo.mp4` (1:45; public link to be added at submission)

1. Summary

Residents, newcomers and front-desk staff of Swiss municipalities ask the same questions every day (AHV contributions, registering after a move, health insurance deadlines, voting by post, taxes, emergency numbers) in four languages. Public administrations cannot send citizen questions to foreign cloud LLMs. SwissDoc Q&A answers such questions **only from an administration's own documents**, in the user's language (DE/FR/IT/EN), with **source citations**, and **refuses** out-of-scope questions. The whole stack (Apertus inference server plus web app) starts with `make run` and runs **with no internet connection at runtime**. On our 26-question evaluation set (22 answerable, 4 out-of-scope), the default CPU setup with Apertus-v1.1-0.5B-Instruct reaches 100% retrieval hit@3, 86.4% answer fact accuracy, 90.9% citation accuracy and 100% correct abstention, at about 6 s per answer on an 8-core CPU without a GPU.

2. Architecture



- **app** (`src/swissdoc/`): corpus loader and chunker, a dependency-free multilingual BM25 retriever (accent folding, DE/FR/IT/EN stop words, prefix stemming), RAG orchestration, a web UI and a JSON API (`POST /api/ask`, `GET /healthz`). It has zero third-party runtime dependencies.
- **llm** (`src/llm_server.py`): a minimal OpenAI-compatible server (`/v1/models`, `/v1/chat/completions`) that runs Apertus with Hugging Face transformers on CPU or GPU. Any other OpenAI-compatible Apertus endpoint (vLLM, llama.cpp, a Swiss cloud) can replace it via `LLM_BASE_URL`.
- **Robustness:** if the LLM is unreachable, the app degrades to an *extractive* mode that returns the best-matching source sentences verbatim with citations. If the LLM output fails the grounding guard, the app returns the extractive answer instead (mode `guarded`).

Target architecture (mandatory)

SwissDoc is deployable in **all three** architectures. The default `make run` demonstrates **(b) air-gapped**:

- **b) Air-gapped:** the `llm` container is attached only to a Docker network declared `internal: true`, so it has no route to the internet. The model weights are baked into the image at build time, and `HF_HUB_OFFLINE=1` / `TRANSFORMERS_OFFLINE=1` are set. The app container serves the UI on `127.0.0.1:8080` only. Images can be built on a connected machine, exported with `docker save`, and loaded inside the air-gapped zone.
- **a) On-premise:** the same compose file runs on any Linux host with Docker. For larger loads, point `LLM_BASE_URL` at an on-prem vLLM server running `swiss-ai/Apertus-v1.5-8B` (`make run-external`).
- **c) Sovereign Swiss cloud:** both containers are standard OCI images and can run on any Swiss-operated provider, such as a Kubernetes or VM offering in Swiss data centres. No data leaves the deployment.

External dependencies. *Build time:* Docker Hub `python:3.11-slim`, PyPI (torch CPU, transformers, huggingface_hub) and the Hugging Face Hub for the Apertus weights. *Runtime:* none.

3. Use of Apertus

- **Model (default):** `swiss-ai/Apertus-v1.1-0.5B-Instruct` (Apache-2.0), unquantised FP32 on CPU, about 1 GB of weights. **Also tested:** `swiss-ai/Apertus-v1.1-1.5B-Instruct`, selectable with `APERTUS_MODEL_ID=...`. **Recommended for production:** `swiss-ai/Apertus-v1.5-8B` served by vLLM on a GPU, connected via `make run-external`.
- **How it is used:** inference. Grounded generation (RAG) of short answers in the user's language, with citations.
- **Where it runs:** local weights inside the `llm` container (CPU by default, CUDA used automatically if available).

- **Prompt:** the system prompt tells the model to answer only from the provided sources, cite source ids in square brackets, say "don't know" when the sources lack the answer, use at most 3 sentences, and answer in the detected language. The user turn contains the top-3 sources (`[id] title + text`) followed by the question. Decoding uses temperature 0.1 and up to 256 new tokens, with the model's own chat template.
- **Why Apertus:** it is fully open (weights, data and recipes), trained with respect for data-owner opt-outs, multilingual with strong coverage of the Swiss national languages, and developed in Switzerland (ETH Zurich and EPFL). Those properties make it a good fit for public administrations that must justify their AI supply chain.
- **No other LLMs** were used, either in the product or as judges. Evaluation is fully rule-based (see §5).

4. Data

`data/corpus/` holds 12 short documents (about 7 KB in total) in DE, FR, IT and EN. They cover AHV/AVS, voting by post, residence permits and registration, mandatory health insurance, waste disposal, tax returns and emergency numbers. The team **wrote them as plain-language summaries** of generally known Swiss public-service rules, licensed CC-BY-4.0. They contain no personal data. They are illustrative only, not legal advice, and some details vary by canton. A real deployment would replace them with the administration's own official documents: drop Markdown files with a small front-matter header (`id`, `title`, `lang`) into `data/corpus/`. `eval/questions.jsonl` contains the 26 evaluation questions, written by the team.

5. Evaluation

Task: answer 22 in-scope questions (all four languages, with 6 that need a specific number or date) and refuse 4 out-of-scope ones (Bitcoin price, football, fondue recipe, Python programming).

Metrics: retrieval hit@3 (the expected document is among the retrieved sources). *Fact accuracy:* the answer contains the expected key fact (string match, with synonyms such as "5"/"fünf"). *Citation accuracy:* the answer cites the expected document. *Abstention accuracy:* out-of-scope questions are refused.

Baseline: the same pipeline with no LLM (extractive, verbatim source sentences). **Hardware:** 8 vCPU, 16 GB RAM, no GPU.

Setup	Hit@3	Fact acc.	Citation acc.	Abstention	Latency/q
Extractive baseline (no LLM)	1.00	0.909	1.00	1.00	<0.01 s
Apertus-v1.1-0.5B-Instruct, no guard	1.00	0.864	0.864	1.00	5.7 s
Apertus-v1.1-0.5B-Instruct + guard (default)	1.00	**0.864**	**0.909**	1.00	6.0 s
Apertus-v1.1-1.5B-Instruct + guard	1.00	0.818	0.682	1.00	4.5 s

Findings. (1) Retrieval and the score-threshold abstention are reliable on this corpus. Out-of-scope questions never reach the LLM, which removes a whole class of hallucinations. (2) On this small, lexically aligned set, verbatim extraction scores slightly higher on fact matching. Apertus adds what extraction cannot: fluent, short answers in the user's language that synthesise across sentences. (3) The tiny on-device models still sometimes pick the wrong number from the context (for example, Rega 143 instead of 1414). The guard catches numbers that are absent from the sources, but not wrong numbers that do appear in them. This is why every answer shows its sources, and why we recommend Apertus-v1.5-8B behind vLLM for production. (4) The 1.5B model often wrote the source title instead of the bracketed id, which lowered its citation score. The app then attaches the top source automatically. Raw per-question outputs are in `eval/results_*.json`.

Caveats. The evaluation set is small and team-written, the abstention threshold (2.0) was tuned on the same set, and string-match scoring can under-count correct paraphrases.

6. Limitations

- The corpus is small and illustrative. Lexical BM25 retrieval struggles with paraphrases that share no vocabulary with the sources, and cross-language retrieval only works through shared stems (for example "AHV"/"AVS" do not match). A multilingual embedding model would fix this.
- The 0.5B/1.5B models are weak at selecting the right fact among several similar ones. Use the 8B model for production.
- The grounding guard only checks numbers and question echoes. It does not detect semantically wrong statements.
- Language detection uses a keyword heuristic. Very short questions default to English.

- CPU latency of about 5–6 s per answer, serialised through one model lock, is fine for a front desk or kiosk but not for high load. vLLM scales this.
- There is no authentication. The UI binds to localhost only, so put it behind the organisation's reverse proxy or SSO before exposing it.
- Not evaluated: Romansh, adversarial prompt injection inside documents, or long documents.

7. Reproducibility

- `make run` builds both images (the first build downloads about 1 GB of weights and the torch CPU wheel) and starts the stack. The UI is at http://localhost:8080. Verified on Docker 26.1 with Compose v2.26 on Linux x86-64 (8 vCPU, 16 GB RAM, no GPU).
- Environment variables: `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`. `APERTUS_MODEL_ID` selects the model baked into the bundled server.
- Other endpoint: `make run-external LLM_BASE_URL=http://<host>:8000/v1 LLM_NAME=swiss-ai/Apertus-v1.5-8B`. For example, serve it with `vllm serve swiss-ai/Apertus-v1.5-8B` on a GPU host.
- Tests: `make test` runs 13 unit and API tests with a fake LLM, no model needed. Evaluation: start an endpoint, then run `make eval`. Decoding uses temperature 0.1 (`do_sample` off below 0.05 in the bundled server), so results can vary slightly between runs.

8. Next steps

- Multilingual dense retrieval plus a reranker, and ingestion of PDF/HTML from official cantonal and municipal sites, with automatic re-indexing.
- Apertus-v1.5-8B on vLLM with measured throughput, plus LoRA fine-tuning on public-administration Q&A for citation discipline.
- An entailment-based faithfulness check, a feedback button for staff, and Romansh support.
- A pilot with one municipality's front desk, measuring time saved per enquiry.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Swiss AI Initiative. *Apertus: Democratizing Open and Compliant LLMs for Global Language Environments.* 2025. https://huggingface.co/swiss-ai
- swiss-ai/Apertus-v1.1-0.5B-Instruct and swiss-ai/Apertus-v1.5-8B model cards, Hugging Face.
- Robertson, S. and Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond.* 2009.
- Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020.